

Projeto Trainee - EDRA(Controle e sistemas embarcados)

1 Instruções da avaliação

O objetivo central deste processo é aferir as suas competências em engenharia de software, a capacidade de resolução de problemas complexos e o trabalho colaborativo em um ambiente técnico.

Estrutura do Projeto

O projeto deverá ser desenvolvido em **duplas** e consiste em dois desafios interligados, cada um focado em uma área crítica da robótica autônoma:

- A. Desafio de Visão Computacional:** Focado em percepção, processamento de imagens e localização de alvos.
- B. Desafio de Controle e Estratégia:** Focado na lógica de movimento, planejamento de trajetória e tomada de decisão.

Requisito de Avaliação

Para a avaliação das soluções e metodologias adotadas, será exigida a elaboração de um **Relatório Técnico** detalhado.

2 Relatório

Deverá ser elaborado um Relatório Técnico dos desafios propostos de, no máximo, 15 páginas (incluindo capa e referências), no qual deverão constar:

- A.** justificativa das metodologias adotadas;
- B.** Descrição das principais decisões de arquitetura e implementação tomadas durante o desenvolvimento;
- C.** Apresentação dos maiores obstáculos técnicos encontrados e a solução específica adotada para superá-los.
- D.** Explicação das soluções finais implementadas, demonstrando como ela atende aos desafios propostos;
- E.** Uma tabela que mostre a contribuição e as responsabilidades de cada membro da dupla em relação às tarefas de cada desafio.

É necessário incluir um link que leva para o repositório ou notebook com os códigos implementados. Não é necessário incluir no relatório os códigos dos algoritmos desenvolvidos.

Não existem requisitos de formatação padrão para o relatório, porém sugere-se utilizar: espaçamento duplo entre linhas, digitadas em papel A4 utilizando a fonte Times New Roman em tamanho 12 (com espaçamento de caracteres Normal). As margens recomendadas são: 2,5 cm à esquerda, 1,25 cm na superior, 1,25 à direita e 1,25 cm na inferior.

3 Desafio 1: Detecção de Formas Geométricas Utilizando Visão Computacional

3.1 Contexto

Este desafio visa o estudo e aplicação de conceitos fundamentais de Visão Computacional Clássica utilizando a biblioteca OpenCV.

A percepção visual é essencial em sistemas de robótica e automação. Neste desafio, você deverá construir um sistema de visão que simula a capacidade de um drone de **identificar e diferenciar objetos** no ambiente com base em suas características de cor e forma.

O sistema deve processar um **feed** de vídeo ou imagens para detectar formas geométricas simples coloridas. O sucesso da detecção dependerá do seu domínio sobre técnicas de pré-processamento, segmentação por cor e análise de contornos.

3.2 Objetivo

Desenvolver um programa em Python com OpenCV para processar imagens, detectando, localizando e tipificando formas geométricas básicas (círculo, quadrado, cruz, etc.) coloridas.

3.3 Requisitos Funcionais

- A. Parametrização do Algoritmo:** Todos os fatores cruciais do algoritmo (limites HSV para segmentação, tamanho do kernel de filtro, argumentos de morfologia, etc.) devem ser definidos em um único local no código para fácil ajuste.
- B. Interface de Usuário (UI) Básica:** Implementar elementos de GUI do OpenCV (como `sliders` ou `trackbars`) para permitir o **ajuste em tempo real** dos parâmetros de filtros e limites HSV, otimizando a detecção de forma interativa.
- C. Visualização:** Exibir o vídeo processado (com o contorno da forma, tipo e cor identificados), garantindo que as detecções sejam visíveis em tempo real (ou próximo a ele) durante a reprodução do vídeo.

3.4 Anexo

Segue em anexo o [link do Drive](#), onde vocês encontrarão um video fornecido com multiplas formas e imagens de formas geometricas fotografadas pelo drone.

4 Desafio 2: Coleta de tesouros com TelloSim

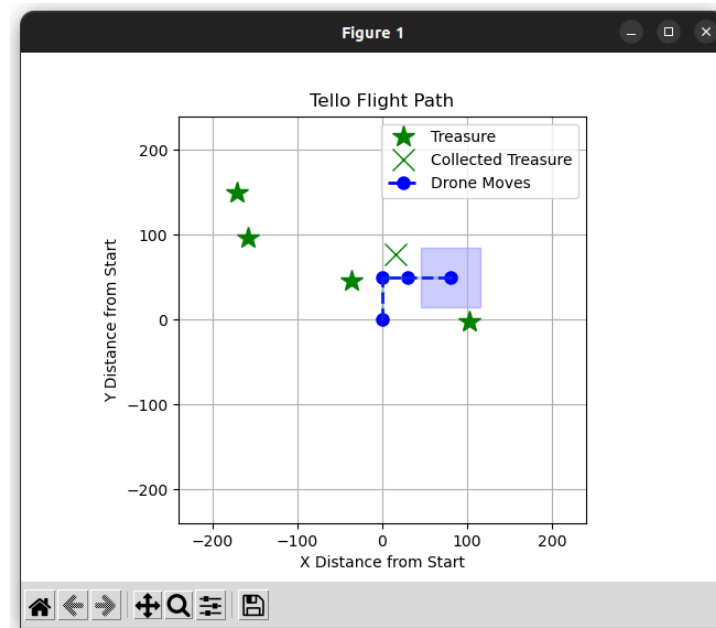


Figura 1: Simulador TelloSimEDRA

4.1 Contexto

Você é responsável por programar um drone de treinamento simulado usando a linguagem Python e a biblioteca chamada TelloSim (adaptado para esse projeto). O drone decola de uma posição inicial (0,0) em um campo limitado de ± 200 cm em X e Y, onde 5 tesouros estarão distribuídos aleatoriamente a cada execução do código. O drone possui uma "área de visão" quadrada de 70x70 cm, que permite coletar tesouros quando a trajetória ou a posição do drone passa sobre eles.

4.2 Objetivo

Programar uma missão de voo que permita ao drone coletar todos os tesouros no campo. O drone deve otimizar a rota para visitar cada tesouro no grid, respeitando a área de visão e evitando movimentos desnecessários, após coletar todos os tesouros o drone deverá retornar para a posição inicial (0,0).

4.3 Regras

- A posição dos tesouros muda a cada execução do simulador.

- O drone deve iniciar em (0,0) e decolar antes de qualquer movimento horizontal.
- A missão será dada como 100% concluída apenas quando todos os tesouros forem coletados e o drone tiver pousado no ponto inicial (0,0).
- O drone pode usar as funções de movimento disponíveis no simulador [dist em cm][deg em graus] :
 - forward(dist), back(dist), left(dist), right(dist)
 - cw(deg), ccw(deg)
 - up(dist), down(dist)
 - takeoff(), land()
- O drone pode usar as funções de status:
 - cur_loc ; [posição atual do drone (x, y)]
 - visited_treasures ; [índices dos tesouros já coletados]
 - treasures ; [número total de tesouros]

4.4 Anexo

Segue em anexo o [link do Drive](#), onde vocês encontrarão o código base para dar início ao projeto. Siga as instruções do README no repositório [tello_sim_EDRA](#) para as instalações e configurações necessárias.

Brasília, 26 de outubro de 2025.